

JavaScript Fundamentals Assessment

Instructions: Welcome to the JavaScript Fundamentals Assessment. This sheet contains four individual assessment tracts. Each tract contains exactly five questions tailored to evaluate your foundational understanding of core programming syntax, logic building, and functional execution in JavaScript. Read the descriptions carefully and write your solutions below each question.

Tract 1: Variables, Syntax & Block Logic

This tract focuses on memory declarations, data representation, basic conditions, and loop structures.

Question 1

What is the fundamental difference between `let`, `const`, and `var` when declaring a variable in terms of reassigning values and block-level visibility?

Question 2

Write a code snippet that declares two primitive data type variables—one string containing a username and one number containing an age. Then, write an `if/else` condition that logs a custom message to the console if the age is 18 or older, and a different message if they are younger.

- **Sub-Question:** What happens to the execution logic if you change the age variable to a string containing the number (e.g., "18") and use a strict inequality operator (`!==`)?

Question 3

Coding Assignment: Array Manipulation Flow

Write a functional JavaScript utility named `buildScoreRegistry()` that manages student scores using fundamentals. Your code must perform the following actions:

- Create an array containing five starting integer test scores.
- Use a native array method to add a new score to the end of the array.
- Use a native loop (such as a `for` or `while` loop) to iterate through the scores and log each value.

Sub-Questions:

1. Write the complete operational code block for this setup.
2. Which array method would you use if you wanted to remove the very first score from the front of your array instead of adding one to the end?
3. How does changing a variable declared with `const` to `let` affect your ability to reassign the entire array to a new empty list later in the code?

Question 4

Coding Assignment: Dynamic Profile Generator

Create a JavaScript function called `formatUserAccount(firstName, lastName, baselineScore)`. The function needs to return a plain object containing:

- A property `fullName` combining both names.
- A property `score` storing the `baselineScore`.
- A simple method inside the object named `upgradeScore()` that increases the object's interior `score` property by exactly 10 points whenever it is called.

Sub-Questions:

1. Write out the full object factory function implementation.
2. Inside the `upgradeScore` method, what keyword must precede the property name to ensure you are modifying the correct object instance's score?
3. If you run this function twice to create `userA` and `userB`, does calling `userA.upgradeScore()` accidentally change the score value inside `userB`? Explain why.

Question 5

Advanced Challenge Puzzle: The Hidden Prototype Reference Trap

Analyze this advanced implementation and decipher its irregular execution mechanics:

```
const accountBase = { score: 50 };
const accessManager = new Proxy(accountBase, {
  get(target, key, receiver) {
    if (key === 'score') {
      return Reflect.get(target, key, receiver) +
this.bonusModifier;
    }
  },
  bonusModifier: 15
});
```

```
Object.defineProperty(Object.prototype, 'bonusModifier', {
  value: 5,
  writable: true,
  configurable: true
});
```

```
console.log(accessManager.score);
```

- **Sub-Question A:** Trace the internal execution context lookup path for `this.bonusModifier` inside the Proxy handler's `get` trap. Why does it resolve successfully instead of throwing a `ReferenceError`?
 - **Sub-Question B:** What exact final number is logged to the console, and how does mutating the global `Object.prototype` directly alter the proxy trap execution?
 - **Sub-Question C:** Predict the structural fallout if `Reflect.get` is invoked with a custom receiver pointing back to a revoked proxy instance.
-

Tract 2: Control Flow, Arrays & Conversions

This tract covers loops, processing array item elements, dynamic type comparisons, and scope errors.

Question 6

What is the operational difference between a standard while loop and a do...while loop regarding the number of times the code block inside is guaranteed to execute?

Question 7

Review the scope configuration block below:

```
function trackInventory() {  
  if (true) {  
    var defaultCount = 50;  
    let dynamicLimit = 100;  
  }  
  console.log(defaultCount);  
  console.log(dynamicLimit);  
}  
trackInventory();
```

- **Sub-Question:** Which specific line triggers an error when this function runs, and what is the exact name of the scope constraint causing that error?

Question 8

Coding Assignment: Target Item Filter

Write a function named `processInventoryList(itemPrices)` that takes an array of numbers representing prices. Your function must achieve the following:

- Create an empty array called `affordableItems`.
- Use a standard loop to check every price element in the input array.
- If a price is less than 50, push it into the `affordableItems` array.
- Return the total count of affordable items.

Sub-Questions:

1. Write the comprehensive script logic for this function.
2. What happens if the input array contains a string element like "40" instead of a number when performing a less-than (<) comparison?
3. How would you rewrite this loop mechanism using the built-in modern `.filter()` array method to achieve the same result in a single line?

Question 9

Coding Assignment: Sentence Builder Template

Write a utility function named `generateStatusMessage(productName, stockQuantity, reorderThreshold)` that processes retail metrics. It must return a text string using template literals

(backticks) that explicitly states the product name, how many units remain, and includes a short boolean status statement showing true or false depending on whether stockQuantity is below the reorderThreshold.

Sub-Questions:

1. Provide the complete code syntax for the function.
2. How do you format variables or expressions inside a template literal string to ensure they evaluate dynamically instead of printing out as literal text?
3. What would happen to your output string if the productName argument is omitted entirely when calling the function?

Question 10

Advanced Challenge Puzzle: Synchronous Iterator Mutation Bounds

Analyze this advanced implementation and decipher its irregular execution mechanics:

```
const sequenceStream = {
  [Symbol.iterator]() {
    let internalStep = 0;
    return {
      next() {
        internalStep++;
        if (internalStep === 1) return { value: internalStep,
done: false };
                                if (internalStep === 2) return { value:
Promise.resolve(internalStep), done: false };
                                return { done: true };
      }
    };
  }
};

(async () => {
  for (const sequenceValue of sequenceStream) {
    console.log(sequenceValue);
  }
})();
```

- **Sub-Question A:** Analyze why a standard synchronous for...of loop processing a synchronous iterator throwing a Promise object does not implicitly pause execution, and contrast this with the specification requirements of a for await...of structure.
 - **Sub-Question B:** What is printed during the second iteration step when executing within this specific asynchronous immediately invoked execution envelope?
 - **Sub-Question C:** Explain how mutating the internal %IteratorPrototype% dynamically during runtime execution could bypass the safety boundaries of standard iteration completion signals.
-

Tract 3: Functions, Scope & Closures

This tract examines parameter returns, functional declarations, object context, and state capture.

Question 11

What is the core syntactic difference between a classic function declaration (using the function keyword) and an arrow function expression (=>) regarding how they handle the this keyword context?

Question 12

Review the timing context block below:

```
const simpleGreeting = {
  username: "Sam",
  sayHi: function() {
    console.log("Hi " + this.username);
  },
  sayLater: function() {
    setTimeout(function() {
      console.log("Later " + this.username);
    }, 100);
  }
};
simpleGreeting.sayHi();
simpleGreeting.sayLater();
```

- **Sub-Question:** Why does the sayLater function fail to print "Later Sam" and output undefined instead? What execution context changed inside the callback?

Question 13

Coding Assignment: Math Operation Factory

Write a JavaScript function called makeMultiplier(multiplierValue). This function should leverage fundamental scopes by returning an entirely separate inner function. The inner function must take a single number parameter, multiply it by the original multiplierValue, and return the final product.

Sub-Questions:

1. Provide the complete implementation code for this factory setup.
2. If you define const double = makeMultiplier(2);, what value is returned when you call double(15)?
3. What is the technical name given to an inner function that remembers and retains access to variables from its outer parental scope environment even after the parent function has finished running?

Question 14

Coding Assignment: Encapsulated Step Tracker

Write a utility named `createStepTracker()` that initializes an isolated variable counter set to 0. It must return an object containing exactly two methods:

- `walk()`: Increments the counter by 1 and returns the current total steps.
- `reset()`: Returns the counter back to 0.

Ensure the counter variable itself cannot be accessed or modified directly from outside the tracker functions.

Sub-Questions:

1. Write out the full script layout for this state tracking utility.
2. If you initialize two distinct trackers called `trackerOne = createStepTracker()` and `trackerTwo = createStepTracker()`, does taking steps on the first one alter the value of the second?
3. How could you alter this factory setup to add a method that allows changing the counter to a new number only if the caller knows a correct text password parameter?

Question 15

Advanced Challenge Puzzle: Context Forwarding Execution Frame

Analyze this advanced implementation and decipher its irregular execution mechanics:

```
const primarySystem = {
  generateCallback() {
    return () => () => console.log(this.systemID);
  },
  systemID: 'SYS-PRIMARY'
};

const foreignSystem = { systemID: 'SYS-FOREIGN' };

const detachedLink =
primarySystem.generateCallback.call(foreignSystem);
const evaluationStep = detachedLink();
const finalizedExecution = evaluationStep.bind(primarySystem);

finalizedExecution();
```

- **Sub-Question A:** Determine the text string printed by evaluating `finalizedExecution()`. Explicitly map out why the subsequent call to `.bind(primarySystem)` completely fails to change the inner execution frame's lexically scoped `this` references.
- **Sub-Question B:** Trace the absolute internal pointer path of the scope chain from the moment `primarySystem.generateCallback.call(foreignSystem)` evaluates through the double arrow initialization sequence.
- **Sub-Question C:** How would rewriting this block using a strict mode directive ("use strict";) at the global level alter the evaluation if the function was originally passed directly as a detached callback reference to a global caller?

Tract 4: Asynchronous Fundamentals & Event Loops

This tract explores fundamental Promise mechanics, execution ordering, async/await flow, and loop runtime safety.

Question 16

What are the three potential statuses that a standard JavaScript Promise object can exist in during its processing lifecycle?

Question 17

Review the order of operations in the script below:

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout Task");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise Task");
});

console.log("End");
```

- **Sub-Question:** Write the precise order of lines printed to the console. What type of queue execution ordering causes the Promise text block to display before the Timeout text block?

Question 18

Coding Assignment: Delayed Resolution Pipeline

Write a function named `delayMessage(text, delayTime)` that handles basic asynchronous operations. Your function must return a standard JavaScript Promise. Inside the Promise, use `setTimeout` to resolve the Promise with the input text message only after the specified `delayTime` (in milliseconds) has passed.

Sub-Questions:

1. Write the comprehensive operational code for this promise wrapper utility.
2. How would you consume this function using modern `async` and `await` keywords inside an execution wrapper to print the message to the console safely?
3. What keyword modifier must be placed immediately in front of a function definition context to allow the internal use of the `await` keyword?

Question 19

Coding Assignment: Basic Error Catch Architecture

Write a functional async function called `fetchDataSafely(dataPromise)`. The utility must use a try/catch block to handle the resolution of `dataPromise`. If the promise resolves successfully, return the result. If it rejects with an error, catch that exception and return a default backup string: "Error: Request could not be processed".

Sub-Questions:

1. Provide the complete script structure for this safe processing routine.
2. If you pass a promise that rejects instantly via `Promise.reject("Network down")` into your function, what exact string will be returned by your routine?
3. How would the processing behavior change if you accidentally omitted the `await` keyword directly in front of the `dataPromise` statement inside the try block?

Question 20

Advanced Challenge Puzzle: Microtask Queue Starvation Invariants

Analyze this advanced implementation and decipher its irregular execution mechanics:

```
let trackingActive = true;

setTimeout(() => {
  trackingActive = false;
  console.log("Timeout task executed successfully!");
}, 10);

function continuousLoop() {
  if (!trackingActive) return;
  Promise.resolve().then(continuousLoop);
}

continuousLoop();
```

- **Sub-Question A:** Explain why the message "Timeout task executed successfully!" will never print under standard ECMAScript event loop architectures, explicitly tracking how the microtask queue behaves when it is perpetually fed new tasks.
- **Sub-Question B:** Describe how the JavaScript engine's call stack and execution context queues react to this code, and explain why this pattern causes the browser tab UI layout thread to completely freeze up despite not creating a stack overflow error.
- **Sub-Question C:** How would substituting the recursive `Promise.resolve()` scheduling trick with a `queueMicrotask()` native call vs a `requestAnimationFrame()` macro-scheduler structurally alter the thread allocation semantics and prevent terminal tab freezing?